Tea Covey

6/12/2026

Introduction to Python Programming

Assignment 05

Assignment 5: data processing using dictionaries and exception handling

Introduction

What we're doing in this assignment: *Create a Python program that demonstrates using constants, variables, and print statements to display a message about a student's registration for a Python course*.

What changed from last assignment: T*his program is very similar to Assignment04, but It* ***adds the use of data processing using dictionaries and exception handling*****.**

In module 05, we learned about dictionary collections (including adding and removing data), using dictionaries with file data, working with JSON files, structured error-handling (try-except), and managing code files with network file sharing, cloud file sharing, and using Github.

In this document, we will address the major changes in the script including using JSON files (and its associated .json functions for processing data) and error handling.

1. Using a JSON file

This module, we are changing from .csv files to .json files.

What is JSON?

Before we get into using JSON in our script, what is JSON? I've provided an excerpt from Module 05 below:

> *JSON (JavaScript Object Notation) is a lightweight data-interchange format that is easy for both humans to read and write and for machines to parse and generate. It's widely used for data serialization and communication between a client and a server, as well as for configuration files and data storage. Here's an explanation of JSON files:*

Basic Structure:

- *JSON files consist of key-value pairs (very much like Python dictionarys* [sic] *do!)*
- *Keys are strings enclosed in double quotes.*
- *Values can be strings, numbers, objects, arrays, booleans, or null.*
- *Data is organized using curly braces {} for objects and square brackets [] for arrays.*
- *Commas , separate key-value pairs or elements within an array.*

Here is an example of [sic]

```
{
"Name": "John Doe",
"Age": 30,
"City": "New York",
"IsStudent": false,
"Hobbies": ["reading", "swimming", "coding"]
}
```

In this JSON object, there are five key-value pairs, where "Name," "Age," "City," "IsStudent," and "Hobbies" are keys, and their corresponding values are strings, numbers, booleans, and an array.

(Mod05-Notes.pdf, page 13)

Importing JSON

Because we will be transitioning to using a .json file instead of a .csv file in this script, we need to import JSON. The JSON library is already built into Python but we need to import the module into the script in order to access its additional features (like json.load() and json.dump()).

```
import json
```

When imports are needed, they should be positioned just below the header of the script before any other code.

Reading JSON file data into our script

In Assignment 04, we were prompted to read data from a CSV file into the students two-dimensional list of lists. Below is the code used to execute that.

```
# Present and Process the data
file = open(FILE_NAME, "r")
for row in file.readlines():
    student_data = row.split(",")
    student_data = [student_data[0], student_data[1], student_data[2].strip()]
    students.append(student_data)
file.close()
```

In assignment 05, we are prompted to load the contents of a JSON file into the students two-dimensional list of dictionary rows.

```
# When the program starts, read the file data into a list of lists (table)
# Extract the data from the file
try:
    file = open(FILE_NAME, "r")
    students = json.load(file)
except FileNotFoundError as e:
    print("This file doesn't exist.")
    print("---Technical Information---")
    print(e, e.__doc__,type(e),sep='\n')
    print("Creating a new file...")
    file=open(FILE_NAME, "w")
except JSONDecodeError as e:
    print("File data is invalid.")
    print("---Technical Information---")
    print(e, e.__doc__, type(e), sep='\n')
    print('Resetting file data...')
    file = open(FILE_NAME, "w")
    json.dump(student_data, file)
except Exception as e:
    print("There was an error opening the file.")
    print("---Technical Information---")
    print(e, e.__doc__, type(e), sep='\n')
finally:
    if not file.closed:
        file.close()
```

In each block of code I have highlighted in yellow the rows that handle the processing of the data into the appropriate format and into the students variable.

In Assignment 04, it is more complex and lengthy code to process the data from the CSV file to the students variable.

(For clarity's sake: the only difference between the two lines of code is not just the use of the JSON v. CSV file. The students variable in Assignment 04 is a two-dimensional list of lists and in Assignment 05 it is a two-dimensional list of dictionary rows.)

In Assignment 05, we only need to use the json.load() function in order to process the data from the JSON file into the students variable.

2. Error Handling: try-except

Before this module, if the user ran into an error/exception while running the program, the program would throw an error and terminate the program. This week, we learned about error handling and using the try-except catch blocks to wrap any processes that could

throw an error. This allows us to predict and anticipate potential error messages and handle them accordingly instead of ending the entire program for the user.

Loading file data into the script

The first example of this is when loading the file data into the script memory.

```
# When the program starts, read the file data into a list of lists (table)
# Extract the data from the file
try:
    file = open(FILE_NAME, "r")
    students = json.load(file)
except FileNotFoundError as e:
    print("This file doesn't exist.")
    print("---Technical Information---")
    print(e, e.__doc__,type(e),sep='\n')
    print("Creating a new file...")
    file=open(FILE_NAME, "w")
except JSONDecodeError as e:
    print("File data is invalid.")
    print("---Technical Information---")
    print(e, e.__doc__, type(e), sep='\n')
    print('Resetting file data...')
    file = open(FILE_NAME, "w")
    json.dump(student_data, file)
except Exception as e:
    print("There was an error opening the file.")
    print("---Technical Information---")
    print(e, e.__doc__, type(e), sep='\n')
finally:
    if not file.closed:
        file.close()
```

Wrapping our text in the try-except block of code allows us to run code cautiously and catch potential errors that could occur and handle them accordingly (instead of Python throwing an error and ending the program).

First, the code we want to run is wrapped in a “try” block. Then, we write an “except” block for “Exception.” Next, we add an “except” block for any specific errors we could anticipate occurring.

For example, because we are opening a file in this code, we can anticipate the FileNotFoundError occurring if the file does not exist. We can then resolve this issue by writing code into this block that will create the file for the user.

In each “except” block, we can customize how the error messages are presented to the user. First, we should advise the user of the issue in user-friendly language (without technical jargon if possible). We do still want to include the technical jargon after the user-

friendly message. But we can print a “technical information” header before providing the technical information so that it improves the readability for the user.

Another important note with try-except blocks is that you have to be careful to ensure your file is closed when using open functions. If we included a file.close() line of code at the bottom of the “try” block but an error occurred in the json.load line of code, our code would never reach the file.close() line and our file would remain open.

To avoid this, we can include a “finally” block that tells Python, if the file is not closed, close the file.

User data entry error handling

The next code we wrapped for error handling is for user input.

```
# Input user data
if menu_choice == "1":  # This will not work if it is an integer!
    try:
        student_first_name = input("Enter the student's first name: ")
        if not student_first_name.isalpha():
            raise ValueError("First name can only contain alphabetic characters.")
        student_last_name = input("Enter the student's last name: ")
        if not student_last_name.isalpha():
            raise ValueError("Last name can only contain alphabetic characters.")
        course_name = input("Please enter the name of the course: ")
        student_data =
{"FirstName":student_first_name,"LastName":student_last_name,"CourseName":course_name
}
        students.append(student_data)
        print(f"{student_first_name} {student_last_name} is now registered for
{course_name}.")
    except ValueError as e:
        print(e)
        print("Invalid entry. Please try again...")
        print("---Technical Information---")
        print(e, e.__doc__, type(e), sep='\n')
    except Exception as e:
        print("There was an issue with your entry.")
        print("---Technical Information---")
        print(e, e.__doc__, type(e), sep='\n')
    continue
```

We can anticipate that the user may enter data that is invalid and we want to prevent accepting invalid entries. For example, if a user is prompted to enter their first name but instead, they enter their phone number, we can prevent accepting and storing this information and, instead, tell the user why their entry was invalid (entry for first name

should only include alphabetic characters). Instead of moving on to the next line of code, it bumps the code back to the original menu output for the user to try again. The user can then select to input data and try again with an understanding of what data they can enter.

Saving data to the file

```
# Save the data to a file
elif menu_choice == "3":
    try:
        file = open(FILE_NAME, "w")
        json.dump(students, file, indent=2)
        print("The following data was saved to " + FILE_NAME)
        for student in students:
            print(f'{student["FirstName"]} {student["LastName"]} is registered for
{student["CourseName"]}')
    except TypeError as e:
        print("JSON data was malformed.")
        print("---Technical Information---")
        print(e, e.__doc__, type(e), sep='\n')
    except Exception as e:
        print("There was an error saving data to the file.")
        print("---Technical Information---")
        print(e, e.__doc__, type(e), sep='\n')
    # Check if a file object exists and is still open
    finally:
        if not file.closed:
            file.close()
    continue
```

The last example of using the try-except error handling code is when saving the data to the file. Again, we wrap the code we want to execute in a try block so that if an error is raised, we can mitigate it and prevent the program from throwing an error and ending the program for the user.

As in each prior try-except blocks, we include an except block for “Exception.”

We can also anticipate that if the data is an inappropriate type for the function or operation applied, Python may throw a TypeError.

To learn more about built-in exceptions, you can hover over any of the exceptions and PyCharm provides a brief explanation and a link to python.org for more information.

For example, if we hover over “TypeError,” PyCharm shows the following information shown in Figure 1.1 below.

Figure 1.1: Hovering over "TypeError" in PyCharm

The link to python.org provides thorough documentation on the exception TypeError. (external link: https://docs.python.org/3.14/library/exceptions.html#TypeError)

Again, as noted earlier when reading the file into the script, we must ensure that a file has been closed when we have opened it in a try-except block. The finally block is again used to make sure that if the file is not closed, the file.close() code will now execute to close our file.

Summary

Assignment 05 builds on the very similar script of Assignment 04 but adds a few changes and new concepts including error handling including try-except blocks, the implementation of JSON files, dictionary data types, and the JSON specific functions that simplifies loading data from a file and saving data back to the file (i.e. json.load() and json.dump()).